Anexo 4A

Tipos en React (JSX → TSX)

Curso: TypeScript

150 min

Objetivos de Aprendizaje

Parte 1 - Tipos Avanzados de TypeScript:

- Aplicar Utility Types (Partial, Omit, Pick, Record)
- Implementar Generic Constraints con extends
- Utilizar Template Literal Types
- Aplicar Mapped Types
- Comprender Type Inference
- Usar el operador satisfies

Parte 2 - React + TypeScript:

- Migrar componentes de JSX a TSX
- Tipar props y children (ReactNode)
- Tipar state con useState<T>
- Tipar eventos de formulario
- Usar hooks con tipos (useRef, useCallback, useMemo)
- Crear componentes genéricos con constraints
- Aplicar React 18 con useId

1. Contexto: Tipos avanzados

15 minutos

El problema: Tipos inflexibles

```
interface Task {
  id: string;
  title: string;
  status: 'pending' | 'inProgress' | 'completed';
}

// ¿Cómo creo un tipo para tarea PARCIALMENTE editable?
// ¿Cómo creo un tipo sin el campo id?
// ¿Cómo creo un tipo solo con los campos de búsqueda?
```

Solución: Utility Types

```
type TaskPartial = Partial<Task>;     // Todos los campos opcionales
type TaskSinId = Omit<Task, 'id'>;    // Sin el campo id
type TaskPreview = Pick<Task, 'id' | 'title'>; // Solo algunos campos
type TaskMap = Record<string, Task>;  // Objeto con claves string
```

2. Utility Types

25 minutos

Partial, Required, Omit, Pick

Utility	Uso
Partial<T>	Todos los campos opcionales
Required<T>	Todos los campos obligatorios
Omit<T, K>	Excluir campos
Pick<T, K>	Seleccionar campos
Record<K,V>	Índice/Map

Por qué usar Utility Types - DRY

Problema: Definir tipos similares manualmente causa duplicación

```
// SIN utility types - duplicación!
interface TaskEdit { title?: string; ... }
interface TaskCreate { title: string; ... }

// CON utility types - DRY!
type TaskEdit = Partial<Task>;
type TaskCreate = Omit<Task, 'id'>;
```

Beneficio	Descripción
Una fuente de verdad	Cambias Task, todo se actualiza
Menos bugs	No redefines tipos incorrectamente
Mantenimiento	Un cambio actualiza 5 tipos derivados

Lab: Utility Types

1. Crear tipo para formulario de edición (Partial)
2. Crear tipo para respuesta sin id (Omit)
3. Crear índice de tareas por prioridad (Record)

Código del Lab

```
// 1. Formulario de edición - todos los campos opcionales
type EditTaskParams = Partial<task>;

// 2. Datos para crear tarea (servidor genera id)
type CreateTaskParams = Omit<task, 'id'="">;

// 3. Índice de tareas por prioridad
type TasksByPriority = Record<string, task[]="">;

// 4. Búsqueda con resultados preview
type TaskSearchResult = Pick<task, 'id'="" |="" 'title'="" 'status'="">;</task,></string,></task,></task>
```

3. Generic Constraints

20 minutos

extends para limitar genéricos

```
function getProp<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const task = { title: 'Learn TS', priority: 'high' };
getProp(task, 'title');    // ✅ string
getProp(task, 'priority'); // ✅ string
getProp(task, 'status');   // ❌ Error: no existe
```

extends vs any - Compile Time Errors

`any` permite cualquier cosa - sin seguridad

```
// SIN constraint - any
function getValue(obj: any, key: any) {
  return obj[key]; // ✅ compila, ❌ puede fallar en runtime!
}

// CON extends - type safety
function getValue<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key]; // ✅ compila SI y solo SI key existe
}
```

Situación	Solución
Debe tener ciertas props	extends { prop: type }
Debe ser objeto	extends object
Solo keys válidas	extends keyof T

Lab: Generic Constraints

1. Crear componente genérico con constraint
2. Usar extends para asegurar que tiene id

Código del Lab

```
interface WithId {
  id: string;
}

interface ListProps<T extends WithId> {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
  keyExtractor: (item: T) => string;
}

function List<T extends WithId>({ items, renderItem, keyExtractor }: ListProps<T>) {
  return (
    <ul>
      {items.map(item => (
        <li key={keyExtractor(item)}>{renderItem(item)}</li>
      ))}
    </ul>
  );
}
```

4. Template Literal Types

15 minutos

Crear tipos de strings específicas

```
type APIRoute = `/api/${string}`;
const taskRoute: APIRoute = '/api/tasks'; // ✅
const invalid: APIRoute = '/other';       // ❌

type TaskEvent = 'onClick' | 'onSubmit';
type Handler = { [K in TaskEvent]?: () => void };
```

La Magia: Errores en Compile Time

Template literals detectan errores antes de runtime

```
// SIN template literals - error en runtime
type Handler = { [key: string]: () => void };
const h: Handler = { onClik: () => {} }; // ✅ Compila! ❌ Runtime

// CON template literals - error en compile time
type TaskEvent = 'onClick' | 'onSubmit';
type Handler = { [K in TaskEvent]?: () => void };
const h: Handler = { onClik: () => {} }; // ❌ Error!
```

Caso	Beneficio
Rutas API	Detecta rutas inválidas
Event handlers	Autocomplete + validation
CSS classes	Tailwind typesafe

Lab: Template Literal Types

1. Tipar acciones de tareas
2. Crear tipos para eventos

Código del Lab

```typescript
// Tipos para acciones de tareas
type TaskAction = 'create' | 'update' | 'delete' | 'complete';
type TaskRoute = `/tasks/${TaskAction}`;

// '/tasks/create' ✅
// '/tasks/delete' ✅
// '/tasks/invalid' ❌

// Tipos para eventos de tarea
type TaskEvent = 'onClick' | 'onSubmit' | 'onChange';
type TaskEventHandler = { [K in TaskEvent]?: () => void };

// Tipos para estados
type TaskStatus = 'pending' | 'inProgress' | 'completed';
type StatusHandler = {
  [K in TaskStatus]: () => string;
};
```

5. Mapped Types

15 minutos

Transformar tipos automáticamente

```
type Readonly<T> = { readonly [K in keyof T]: T[K] };
type Optional<T> = { [K in keyof T]?: T[K] };
type Nullable<T> = { [K in keyof T]: T[K] | null };
```

DRY: Transformaciones Automáticas

Mapped Types permiten transformar tipos sin duplicar código

```
// SIN Mapped Types - manual y propenso a errores
interface User { name: string; email: string; }
type ReadonlyUser = { readonly name: string; readonly email: string; }; // duplicado!

// CON Mapped Types - una fuente de verdad
type Readonly<T> = { readonly [K in keyof T]: T[K] };
type ReadonlyUser = Readonly<User>; // se actualiza si User cambia
```

Transformación	Uso
Readonly<T>	Props que no deben mutar
Optional<T>	Formularios de edición
Nullable<T>	API responses

Lab: Mapped Types

1. Hacer todos los campos opcionales
2. Agregar prefijos

Código del Lab

```
// Mapped type para hacer todos los campos opcionales
type Partial<t> = { [K in keyof T]?: T[K] };

// Mapped type para hacer todos los campos readonly
type Readonly<t> = { readonly [K in keyof T]: T[K] };

// Mapped type para hacer todos los campos nullable
type Nullable<t> = { [K in keyof T]: T[K] | null };

// Mapped type con prefijo
type WithPrefix<t, p="" extends="" string=""> = {
  [K in keyof T as `${P}${K & string}`]: T[K]
};

type TaskPrefix = WithPrefix<task, 'task_'="">;
// { task_id: string; task_title: string; ... }</task,></t,></t></t></t>
```

6. Type Inference

10 minutos

Inferencia en funciones

```
// TS infiere el tipo de retorno
function getTask() {
  return { id: '1', title: 'Learn TS' };
}
// Infiere: { id: string; title: string }

// as const - preservar literales
const colors = ['red', 'blue'] as const;
// readonly ['red', 'blue']
```

Inferencia: Cuándo Confiar y Cuándo Ser Explícito

Situación	Inferencia	Por qué
Variables locales	✅ Sí	TS infiere bien
Return de funciones	✅ Sí	TS infiere el tipo exacto
Arrays de objetos	✅ Sí	Infiere cada propiedad
Constantes literales	❌ No	Quiere string, usa as const

```
// EXPLÍCITO cuando necesitas control
function createTask(title: string): Task { ... }

// as const - preserva literales
const STATUSES = ['pending', 'done'] as const;
type Status = typeof STATUSES[number]; // 'pending' | 'done'
```

7. Satisfies Operator

10 minutos

satisfies vs as

```
// as - pierde literal
const c1 = 'debug' as string; // c1: string

// satisfies - mantiene literal + valida
const c2 = 'debug' satisfies 'debug' | 'info' | 'error';
// c2: 'debug' (literal)
```

Decision Matrix: satisfies vs as

Necesitas...	Solución
Validar tipo PERO mantener literal	satisfies
Forzar tipo sin importar el valor	as
Autocomplete de literales	satisfies
Casting a tipo incompatible	as

```
// Caso práctico: validación con inferencia
type Color = 'primary' | 'secondary' | 'success';
const colores = {
  activo: 'primary',
  hover: 'secondary'
} satisfies Record<string, Color>;

// colores.activo es "primary" (literal), no string
usarColor(colores.activo); // ✅ Funciona
```

Lab: Operador Satisfies para el Caso 1r

```
// Caso: Configuración de tareas tipada
type Priority = 'low' | 'medium' | 'high';

const taskConfig = {
  maxTitleLength: 100,
  defaultPriority: 'medium' as const,
  allowedStatuses: ['pending', 'inProgress', 'completed'] as const
} satisfies {
  maxTitleLength: number;
  defaultPriority: Priority;
  readonly allowedStatuses: readonly (keyof Task)[];
};

// ✅ type narrowing: 'medium' literal, no string
type Priority2 = 'low' | 'medium' | 'high';
function setPriority(p: Priority2) {}
setPriority(taskConfig.defaultPriority); // ✅
```

Ejercicio Integrador – Parte 1

Aplicar conceptos al Caso 2r (Biblioteca)

Requisitos

- Utility Types: Partial, Omit, Pick
- Generic Constraints con extends
- Template Literal Types
- Mapped Types
- Satisfies

8. Migración JSX → TSX

15 minutos

De JSX a TSX

```
// TSX - con tipos explícitos
interface BookCardProps {
  titulo: string;
  autor: string;
  año?: number;
}

function BookCard({ titulo, autor, año }: BookCardProps) {
  return (
    <div>
      <h3>{titulo}</h3>
      <p>{autor} {año && `- ${año}`}</p>
    </div>
  );
}
```

ROI: Por qué Migrar a TSX

Aspecto	JSX	TSX
Errores de props	Runtime	Compile time ✅
Autocomplete	Ninguno	Total ✅
Refactoring	Peligroso	Seguro ✅
Documentación	Comentarios	Tipos ✅

```
// JSX - error en runtime!
<BookCard titulo="Libro" /> // ✅ Compila, ❌ falla (titulo vs title)

// TSX - error en compile time!
<BookCard titulo="Libro" /> // ❌ Error: 'titulo' does not exist
```

Lab: Migrar Componente JSX a TSX

```
interface Task {
  id: string;
  title: string;
  description: string;
  status: 'pending' | 'inProgress' | 'completed';
  priority: 'low' | 'medium' | 'high';
}

interface TaskCardProps {
  task: Task;
  onComplete?: (id: string) => void;
}

function TaskCard({ task, onComplete }: TaskCardProps) {
  return (
    <div className="task-card">
      <h3>{task.title}</h3>
      <p>{task.description}</p>
      <span>{task.status === 'completed' ? 'Completada' : 'Pendiente'}</span>
```

9. Tipado de Props y Children

15 minutos

Props con Children

```
import { ReactNode } from 'react';

interface CardProps {
  title: string;
  children: ReactNode;
  footer?: ReactNode;
}
```

ReactNode vs otros tipos

Tipo	Qué acepta
ReactNode	Todo
ReactElement	Solo elementos JSX
string	Solo texto

Props: Cuándo Opcionales vs Requeridas

```
// Props requeridas - el componente NO funciona sin ellas
interface ButtonProps {
  label: string;           // Requerido
  onClick: () => void;   // Requerido
}

// Props opcionales - tienen valor por defecto
interface ButtonProps {
  variant?: 'primary' | 'secondary';  // default 'primary'
  disabled?: boolean;                  // default false
}
```

Lab: Tipar Componentes con Children

```
function Layout({ children, header }: { children: React.ReactNode; header: React.ReactNode }) {
  return (
    <div>
      <header>{header}</header>
      <main>{children}</main>
    </div>
  );
}

interface ButtonProps {
  label: string;
  onClick?: () => void;
  variant?: 'primary' | 'secondary';
  disabled?: boolean;
}

function Button({ label, onClick, variant = 'primary', disabled }: ButtonProps) {
  return (
    <button className={`btn btn-${variant}`} onClick={onClick} disabled={disabled}>
```

10. Tipado de State

15 minutos

useState con genéricos

```
// Inferencia automática
const [task, setTask] = useState({ title: 'Learn' });

// Explícito
const [tasks, setTasks] = useState<Task[]>([]);
const [task, setTask] = useState<Task | null>(null);
```

useState: Inferencia vs Explícito

```
// ❌ EXPLÍCITO redundante
const [name, setName] = useState<string>(''); // string ya se infiere

// ❌ EXPLÍCITO necesario - valor inicial ambiguous
const [user, setUser] = useState<User | null>(null);

// ✅ INFIRME - tipo obvio
const [count, setCount] = useState(0);
const [items, setItems] = useState<string[]>([]);
const [loading, setLoading] = useState(false);
```

Lab: Estado Tipado para Tareas

```
const [tasks, setTasks] = useState<Task[]>([]);
const [loading, setLoading] = useState<boolean>(false);
const [error, setError] = useState<string | null>(null);

function addTask(newTask: Task) {
  setTasks(prev => [...prev, newTask]);
}

function removeTask(id: string) {
  setTasks(prev => prev.filter(t => t.id !== id));
}
```

11. Tipado de Eventos

15 minutos

Eventos de formulario

```
// Input change
onChange={(e) => setValue(e.target.value)}
// e: ChangeEvent<HTMLInputElement>

// Form submit
onSubmit={(e) => e.preventDefault()}
// e: FormEvent<HTMLFormElement>
```

Eventos: Errores en Compile Time

```
// ❌ ERROR: type 'string' is not assignable
<input onChange={(e) => setValue(123)} />

// ✅ CORRECTO: type-safe
<input value={value} onChange={(e) => setValue(e.target.value)} />

// ❌ ERROR: Property 'target' does not exist
<button onClick={() => console.log(e.target)}>

// ✅ CORRECTO: tienes el tipo completo del evento
<button onClick={(e) => console.log(e.currentTarget)}>
```

Lab: Formulario de Tareas Tipado

```
interface TaskFormData {
  title: string;
  description: string;
  priority: 'low' | 'medium' | 'high';
}

function TaskForm({ onSubmit }: { onSubmit: (data: TaskFormData) => void }) {
  const [title, setTitle] = useState('');

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    onSubmit({ title, description: '', priority: 'medium' });
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={title}
        onChange={(e) => setTitle(e.target.value)}
```

12. Hooks Básicos Tipados

15 minutos

useRef, useCallback, useMemo

```
// useRef
const inputRef = useRef<HTMLInputElement>(null);

// useCallback
const handleClick = useCallback(() => { ... }, []);

// useMemo
const filtered = useMemo(() => tasks.filter(...), [tasks]);
```

Hooks: Optimización Real vs Teórica

Hook	¿Cuándo realmente ayuda?	¿Cuándo es overkill?
useMemo	Cálculos pesados en cada render	Valores simples o funciones inline
useCallback	Pass callbacks a child memorizados	Handlers simples sin children
useRef	Access DOM, valores mutables	State que necesita re-render

Lab: Optimizar Componente con Hooks

```
function TaskList({ tasks, filter }: { tasks: Task[]; filter: string }) {
  const inputRef = useRef<HTMLInputElement>(null);

  const filteredTasks = useMemo(() => {
    if (!filter) return tasks;
    return tasks.filter(task =>
      task.title.toLowerCase().includes(filter.toLowerCase())
    );
  }, [tasks, filter]);

  const focusInput = useCallback(() => {
    inputRef.current?.focus();
  }, []);

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Buscar..." />
      <button onClick={focusInput}>Focus</button>
    </div>
```

13. Componentes Genéricos

15 minutos

Componentes con tipo genérico

```
interface ListProps<T> {
  items: T[];
  renderItem: (item: T) => ReactNode;
}

function List<T>({ items, renderItem }: ListProps<T>) {
  return <ul>{items.map(renderItem)}</ul>;
}
```

Componentes Genéricos: Reutilización Real

```
// ❌ Sin genéricos: repetir código
<StringList items={names} />
<NumberList items={ages} />
<UserList items={users} />

// ✅ Con genéricos: UN componente para todos
<List<string> items={names} renderItem={(n) => <li>{n}</li>} />
<List<number> items={ages} renderItem={(a) => <li>{a}</li>} />
<List<User> items={users} renderItem={(u) => <li>{u.name}</li>} />
```

Lab: Componente List Genérico

```
interface Entity { id: string; }
interface Task extends Entity { title: string; description: string; status: string; }

interface DataListProps<T extends Entity> {
  data: T[];
  renderItem: (item: T) => React.ReactNode;
  keyExtractor?: (item: T) => string;
}

function DataList<T extends Entity>({ data, renderItem, keyExtractor = (item) => item.id }: DataListProps<T>) {
  if (data.length === 0) return <div>No hay datos</div>;

  return (
    <ul>
      {data.map(item => (
        <li key={keyExtractor(item)}>{renderItem(item)}</li>
      ))}
    </ul>
  );
```

14. IDs Únicos en Componentes

10 minutos

useId para accesibilidad y SSR

```
const id = useId();
return (
  <div>
    <label htmlFor={id}>Nombre</label>
    <input id={id} />
  </div>
);
```

useId: Accesibilidad y SSR

```
// ❌ ERROR: IDs hardcodeados en bucles (duplicados)
{items.map(item => (
  <>
    <label htmlFor="input">{item.name}</label>
    <input id="input" />
  </>
))}

// ✅ CORRECTO: useId genera IDs únicos por render
{items.map(item => {
  const id = useId(); // ⚠️ No dentro de map (hook rule)
  return (
    <>
      <label htmlFor={id}>{item.name}</label>
      <input id={id} />
    </>
  );
})}
```

Lab: IDs Únicos en Componentes

```
import { useId } from 'react';

interface InputProps {
  label: string;
  type?: 'text' | 'email' | 'password';
  value: string;
  onChange: (value: string) => void;
}

function Input({ label, type = 'text', value, onChange }: InputProps) {
  const id = useId();

  return (
    <div>
      <label htmlFor={id}>{label}</label>
      <input id={id} type={type} value={value} onChange={(e) => onChange(e.target.value)} />
    </div>
  );
}
```

Ejercicio Integrador – Parte 2

Componentes React Tipados - Caso 2r

Requisitos

- BookCard: props tipadas con children
- BookList: componente genérico
- LoanForm: eventos tipados
- useState, useRef, useCallback
- useId para accesibilidad

15. Resumen

5 minutos

Puntos Clave

1. **Utility Types:** Partial, Omit, Pick, Record
2. **Generic Constraints:** extends para limitar tipos
3. **Template Literal Types:** `${}` para strings
4. **Mapped Types:** [K in keyof T]
5. **Type Inference:** Cómo TS infiere tipos
6. **Satisfies:** Validar sin perder literales
7. **JSX → TSX:** Migración de componentes
8. **Props y Children:** ReactNode
9. **State:** useState<T>

10**Eventos:** ChangeEvent, FormEvent

11**Hooks:** useRef, useCallback, useMemo

12**Componentes Genéricos:** <T> con constraints

13**useId:** IDs únicos para accesibilidad

16. Referencias

Recursos Adicionales

- TypeScript Utility Types
- TypeScript Generics
- Template Literal Types
- satisfies Operator
- React TypeScript Cheatsheet
- React 18 with TypeScript
- useId Hook

Gracias!

Anexo 4A Completado

Siguiente: Anexo 5A - SSR